Lab 4: Serve and measure

Hugging Face versus an inference engine

Aarne Talman

19 November 2026

LDA-T318 | LLMs and Generative AI for NLP | University of Helsinki

The question

Lab 4

When is an inference engine worth it, compared with plain Hugging Face?

You run the same model, Qwen3.5-0.8B, in two ways:

- **Hugging Face**: `model.generate()` in your own Python process;
- **vLLM**: a server that handles many requests at once.

For both you run a **speed benchmark** and an **evaluation** on your Lab 2 test set. Then you design **one experiment of your own**.

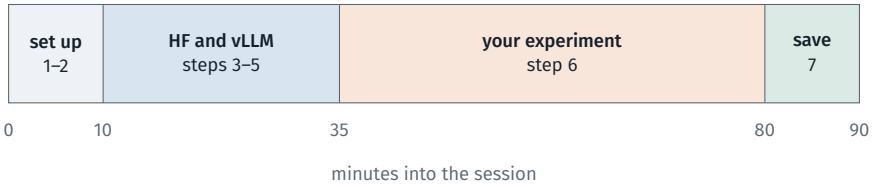
Commands: `Week-4/Lab-4/README.md`, or the same steps in `lab4.ipynb`.

For the final report

- the tables from `summarize_results.py`
- your experiment: a table and a paragraph
- a short answer to the question above

Optional: FP8 weights, JSON with a schema.

Today



Everything runs **interactively** in one GPU session (`srun …--pty bash`). Each command takes seconds to a few minutes, and you see the result straight away.

Start with the **smoke test** (step 1): it checks the scripts with fake models, no GPU. If it has not passed by minute 10, put your hand up.

You do not have to finish today: **continue on your own time**. Work in an SSH session on `roihu-gpu.csc.fi`.

Two ways to run a model, one set of scripts

Hugging Face

`model.generate()` on a padded batch. The whole batch runs until its longest answer is done (*static batching*).

vLLM

A server with an OpenAI-style API. Requests join and leave the batch at every step (*continuous batching*), with a paged KV cache.

	Hugging Face	**vLLM**
speed benchmark	`benchmark.py --backend hf`	`benchmark.py --backend server`
evaluation	`score_task.py --backend hf`	`score_task.py --backend server`
“concurrency 8”	one batch of 8 prompts	8 requests at the same time

Same model, same prompts, same test set and scorer: only the way of running the model changes.

What the numbers mean

Number	What it is	What it shows
time to first token	until the answer starts	prefill
time per output token	the gap between later tokens	decode
tokens per second	output of all requests together	throughput
accuracy	on your Lab 2 test set, Lab 2 scorer	quality

Fair speed test: warm-up requests are not counted, and every request produces exactly 128 tokens, so all requests do the same work.

Accuracy: greedy answers are usually the same from both, but not always. `show_differences.py` lists where they differ.

Your own experiment

Choose **one** question, decide what to vary, run it, and explain the result with the lecture's ideas.

A. Where is the knee? 1, 2, 4 … 64 requests at the same time: when does adding requests stop paying off?

B. Prompt length. Short versus long prompts: what happens to the time to first token, and why?

C. Predict, then measure. Memory bandwidth (4 TB/s) divided by model size (1.6 GB) gives an upper limit for one user. How close do you get, and why not closer?

D. Your own workload. Prompts and answer lengths typical of your task: does vLLM's advantage hold?

E. Your language. Tokens per word for your data in two languages: what does the difference cost in speed and money?

F. Score instead of generate. Pick the label with the highest probability (prefill only): is it as accurate, and how much faster?

What to hand in (in the final report)

The question, what you varied and why, a small table or plot, and a paragraph that explains the result. The README and notebook give a starting point for each option; E and F are for those interested in language technology.

Checklist for today

[ ] files updated and smoke test passed (step 1)
[ ] GPU session started, TASK set (step 2)
[ ] Hugging Face benchmarked and evaluated (step 3)
[ ] vLLM server started, benchmarked, evaluated (step 4)
[ ] tables and differences printed (step 5)
[ ] your experiment run and explained (step 6)
[ ] server stopped, session closed (step 7)

Good practice on Roihu

- an interactive session keeps a whole GPU: leave it with `exit` when you are done
- stop the server first: `bash scripts/stop_server.sh`
- do not run models on the login node

Stuck for more than ten minutes? Put your hand up. The README has a troubleshooting table.